CSCE 5300 – Introduction to Big Data and Data Science

Assignment 3: Coding Data Preprocessing / Data Analysis

Michelle Feliciano

1. Problem Statement

The last digit of my UNT Canvas student login ID is 0. Per the assignment instructions, this value maps to two template categories: "Cleansing 4" (n = 4 or 0) and "1D Visualization" (n = 0, 3, 6, or 9). The objective of this assignment is therefore twofold: (1) write original code that cleans a data set with missing values using the forward-fill / backward-fill technique introduced in Chapter 3, and (2) write original code that produces a 1D line-plot visualization, in the style of Chapter 3's time-series example.

Both parts use one original, connected scenario, unrelated to the course's reindex()/df/one-two-three template example: a mobile hair-styling business that records its total revenue at the end of every month. In a few months the stylist was on the road and never logged a total, so those months arrive in Python as missing values (NaN). Part A cleans those gaps; Part B visualizes the resulting monthly revenue trend across the year.

2. Method

2.1 Cleansing 4 – Fill NA Forward and Backward

Twelve months of revenue are stored in a pandas Series named monthly_revenue_usd, indexed by month abbreviation, with four months set to np.nan to represent unlogged totals. isnull() is used first to identify exactly which months are missing. Then two pandas methods are applied and compared: ffill(), which carries the most recently recorded month's total forward into each gap, and bfill(), which instead borrows the next recorded month's total backward into each gap. The forward-filled result is treated as the "cleaned" series and is saved to a CSV file so Part B can reuse it.

2.2 1D Visualization – Line Plot of a Time Series

The cleaned (forward-filled) monthly revenue series from Part A is loaded and plotted with matplotlib's plt.plot(), following the same idea as Chapter 3's population line-plot example: month labels on the x-axis and revenue totals on the y-axis, with a title and axis labels describing the original data rather than the template's.

3. Code

3.1 Part A – cleansing4_fill_missing_revenue.py

```
# CSCE 5300 - Assignment 3
# Part A: Cleansing 4 - Fill NA Forward and Backward
#
# Original scenario (not from the course template): a mobile hair-styling
# business records its total revenue at the end of every month. A few
# months were never logged (the stylist was on the road and forgot to
# enter the total), so those months come into Python as missing values
# (NaN). This script demonstrates two ways to fill those gaps: carrying
# the last known month's total forward, and borrowing the next known
```

```
# month's total backward.
#
# All variable names, the business scenario, and the data values below
# are original and unrelated to the course's reindex()/df/one-two-three
# template example.

import pandas as pd
import numpy as np

# Twelve calendar months of one operating year.
service_months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                  'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

# Recorded monthly revenue in US dollars. np.nan marks a month whose
# total was never logged.
monthly_revenue_usd = pd.Series(
    [2150.0, np.nan, 2480.0, 2600.0, np.nan, np.nan,
     3100.0, 2950.0, np.nan, 3300.0, 3450.0, 3600.0],
    index=service_months,
    name='monthly_revenue_usd'
)

print("Step 1 - Raw monthly revenue, with unrecorded months as NaN:")
print(monthly_revenue_usd)

# How many months are missing, and which ones.
missing_month_flags = monthly_revenue_usd.isnull()
print("\nStep 2 - Which months are missing?")
print(missing_month_flags)
print(f"\nTotal unrecorded months: {int(missing_month_flags.sum())}")

# Forward-fill: an unrecorded month is assumed to match the most recent
# month that WAS recorded (reasonable if revenue is fairly steady
# month-to-month and the business simply forgot to log a number).
revenue_filled_forward = monthly_revenue_usd.ffill()
print("\nStep 3 - Forward-filled revenue (missing month repeats the prior "
      "recorded month's total):")
print(revenue_filled_forward)

# Backward-fill: an unrecorded month instead borrows the next recorded
# month's total. Shown for comparison against the forward-fill result.
revenue_filled_backward = monthly_revenue_usd.bfill()
print("\nStep 4 - Backward-filled revenue (missing month borrows the next "
      "recorded month's total):")
print(revenue_filled_backward)

# Save the forward-filled series to a small CSV so Part B (the 1D line
# plot) can plot the cleaned trend without recomputing it.
revenue_filled_forward.to_csv('cleaned_monthly_revenue.csv', header=True)
print("\nSaved cleaned (forward-filled) series to cleaned_monthly_revenue.csv")
```

3.2 Part B – visualization_revenue_trend.py

```
# CSCE 5300 - Assignment 3
# Part B: 1D Visualization - line plot of a time series
#
# Original scenario: plot the cleaned monthly revenue trend for the same
```

```
# mobile hair-styling business used in Part A (Cleansing 4), so the
# preprocessing step and the analysis step tell one connected story.
# The variable names and data values are original; only the general
# "line plot of a time series with plt.plot()" idea follows the
# course's population-growth example.

import matplotlib
matplotlib.use('Agg')  # render to file without a display
import matplotlib.pyplot as plt
import pandas as pd

# Load the cleaned (forward-filled) revenue series produced by Part A.
cleaned_revenue = pd.read_csv(
    'cleaned_monthly_revenue.csv', index_col=0
)['monthly_revenue_usd']

operating_months = cleaned_revenue.index.tolist()
revenue_totals_usd = cleaned_revenue.values

plt.figure(figsize=(8, 5))
plt.plot(operating_months, revenue_totals_usd, marker='o',
         color='#1E5631', linewidth=2)
plt.title("Mobile Hair-Styling Business: Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue (USD)")
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig('monthly_revenue_trend.png', dpi=150)
print("Saved line plot to monthly_revenue_trend.png")

print("\nData plotted (cleaned monthly revenue):")
print(cleaned_revenue)
```

4. Results

4.1 Cleansing 4 output

The script below confirms four months (Feb, May, Jun, Sep) were missing, then shows the forward-filled and backward-filled results side by side. Forward-fill and backward-fill agree everywhere except across the missing runs, where each method borrows from a different direction — for example, May and Jun are filled with April's 2600.00 under forward-fill, but with July's 3100.00 under backward-fill.

```
Step 1 - Raw monthly revenue, with unrecorded months as NaN:
Jan    2150.0
Feb       NaN
Mar    2480.0
Apr    2600.0
May       NaN
Jun       NaN
Jul    3100.0
Aug    2950.0
Sep       NaN
Oct    3300.0
Nov    3450.0
Dec    3600.0
```

```
Name: monthly_revenue_usd, dtype: float64

Step 2 - Which months are missing?
Jan    False
Feb     True
Mar    False
Apr    False
May     True
Jun     True
Jul    False
Aug    False
Sep     True
Oct    False
Nov    False
Dec    False
Name: monthly_revenue_usd, dtype: bool

Total unrecorded months: 4

Step 3 - Forward-filled revenue (missing month repeats the prior recorded month's
total):
Jan    2150.0
Feb    2150.0
Mar    2480.0
Apr    2600.0
May    2600.0
Jun    2600.0
Jul    3100.0
Aug    2950.0
Sep    2950.0
Oct    3300.0
Nov    3450.0
Dec    3600.0
Name: monthly_revenue_usd, dtype: float64

Step 4 - Backward-filled revenue (missing month borrows the next recorded month's
total):
Jan    2150.0
Feb    2480.0
Mar    2480.0
Apr    2600.0
May    3100.0
Jun    3100.0
Jul    3100.0
Aug    2950.0
Sep    3300.0
Oct    3300.0
Nov    3450.0
Dec    3600.0
Name: monthly_revenue_usd, dtype: float64

Saved cleaned (forward-filled) series to cleaned_monthly_revenue.csv
```

4.2 1D Visualization output

```
Saved line plot to monthly_revenue_trend.png
```

```
Data plotted (cleaned monthly revenue):
Jan    2150.0
Feb    2150.0
Mar    2480.0
Apr    2600.0
May    2600.0
Jun    2600.0
Jul    3100.0
Aug    2950.0
Sep    2950.0
Oct    3300.0
Nov    3450.0
Dec    3600.0
Name: monthly_revenue_usd, dtype: float64
```

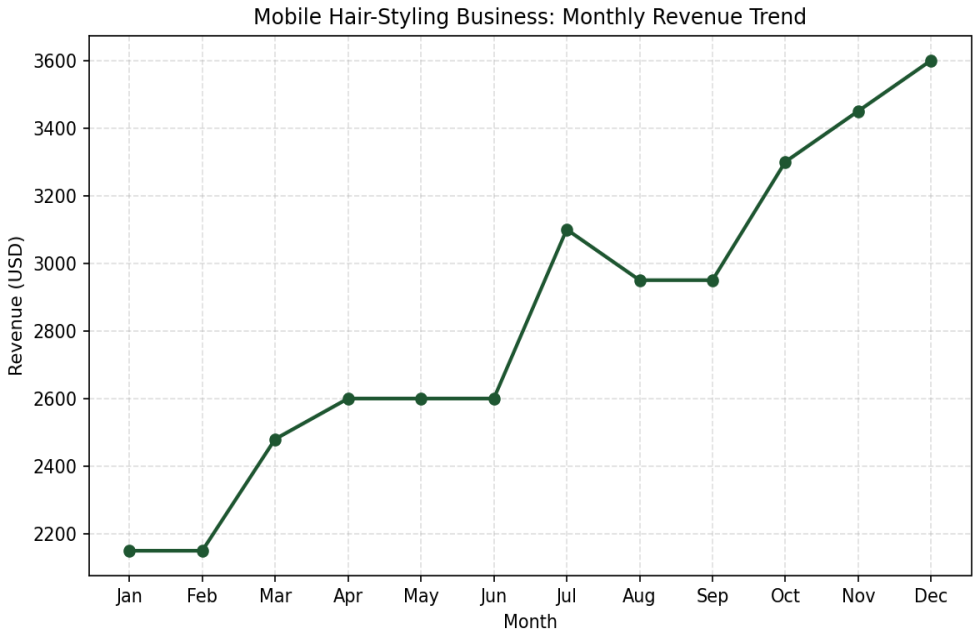


Figure 1: Cleaned (forward-filled) monthly revenue trend for the mobile hair-styling business.

The chart shows revenue rising from about $2,150 in January to $3,600 in December. The forward-fill decision is visible in the plot itself: February holds flat at January's level, and May–June hold flat at April's level, because each of those unlogged months repeats the most recent known total rather than showing a real month-over-month change.

5. Conclusion

This assignment demonstrated two connected steps of the big data analytics pipeline described in Chapter 3: preprocessing and analysis. Cleansing 4 (forward-fill / backward-fill) resolved missing monthly revenue values using two different, clearly distinguishable assumptions, and the 1D line-plot visualization then turned the cleaned series into an interpretable trend. Together they show why data preprocessing has to happen before analysis: the choice of fill method directly changes the shape of the resulting chart, so an analyst should choose — and disclose — that method deliberately rather than treating missing-data handling as an afterthought.